



Institut Teknologi Bandung

Directorate of Sustainable Professional Education

CHAPTER 10

Agen kode

Kode adalah alat yang bisa menjadi alat apa pun — itu sebabnya ia paling berguna, dan itu sebabnya ia satu-satunya yang butuh dinding sungguhan.

Duration 3 jam (2 sesi)

Instructor Hendri Karisma

Source Grootendorst & Alammari, An Illustrated Guide to AI Agents — bab 10

Session Objectives

By the end of this session, participants can:

- 1 **Menjelaskan kenapa menulis kode mengubah sifat jawaban** — dari ditebak jadi dihitung, dan dari tak terbaca jadi bisa diperiksa.
- 2 **Menyebutkan lima jenis alat kode** dan memisahkan yang baca dari yang menulis.
- 3 **Menghitung kenapa seluruh repo tidak muat**, dan menjelaskan pola peta-cari-baca.
- 4 **Menyebutkan lapis sandbox** beserta apa yang TIDAK ditutupnya.
- 5 **Membandingkan alur tetap dengan agen bebas** untuk tugas rekayasa perangkat lunak.
- 6 **Menyebutkan risiko khas** agen kode yang tidak muncul pada agen lain.

SITE

[Course home](#)

REPO

[ai-agentic-demo — kasus code_analysis](#)

BOOK

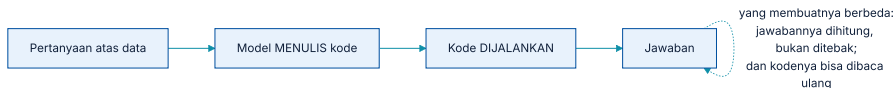
[Maarten Grootendorst & Jay Alamar, An Illustrated Guide to AI Agents \(O'Reilly Media, Early Release\)](#)

01

Kenapa kode

Bukan karena penggunaanya programmer.

Menulis kode mengubah sifat jawabannya



Jawaban yang dihitung, dengan cara menghitung yang bisa dibaca ulang.

Ketika model menjawab pertanyaan data secara langsung, jawabannya adalah **tebakan yang masuk akal**. Ketika ia menulis kode yang lalu dijalankan, jawabannya **hasil perhitungan** — dan cara menghitungnya ada di layar untuk diperiksa.

Ini alasan yang sama kenapa modul ini terus mengulang *serahkan aritmetika ke kode*. Bedanya di sini: **kodenya ditulis saat itu juga**, untuk pertanyaan yang belum pernah ada sebelumnya.

Penggunanya sering justru bukan programmer

Yang dilihat pengguna

\u201cBerapa rata-rata tunggakan nasabah kategori B tahun lalu, dipisah per wilayah?\u201d — lalu sebuah tabel dan grafik.

Yang terjadi

Agen menulis kueri dan beberapa baris pengolahan, menjalankannya, dan menyusun hasilnya. Kodenya tersimpan di jejak.

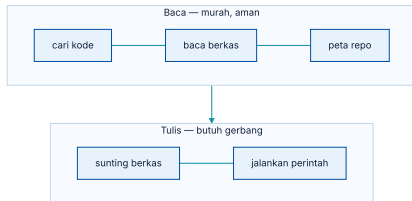
Dan itu yang membuatnya bisa dipertanggungjawabkan: **kalau angkanya dipertanyakan, kodenya bisa dibaca.** Jawaban langsung dari model tidak punya sifat itu.

02

Alatnya

Lima jenis, dan garis yang memisahkan dua kelompok.

Lima jenis alat kode, dan garisnya



Membaca aman dan murah; menulis dan menjalankan tidak.

ALAT	KELOMPOK	CATATAN
Peta repo	Baca	Struktur repo dalam beberapa ribu token
Cari kode	Baca	Berdasarkan nama, simbol, atau teks — bukan kemiripan makna
Baca berkas	Baca	Sebagian berkas, dengan nomor baris
Sunting berkas	Tulis	Butuh gerbang, dan harus bisa dibatalkan
Jalankan perintah	Tulis	Paling berbahaya: ia bisa jadi alat apa pun

Kueri basis data sebagai alat, dan batas yang harus menyertainya

Hanya baca, selalu

Akun terpisah yang benar-benar tidak punya izin menulis. Bukan janji di prompt — izin di basis datanya.

Batas waktu dan baris

Kueri tanpa batas bisa menahan basis data produksi. Batasi di sisi peladen, bukan di kueri.

Tampilan, bukan tabel mentah

Agen melihat tampilan yang sudah menyaring kolom pribadi. Yang tidak terlihat tidak bisa bocor.

Kuerinya masuk jejak

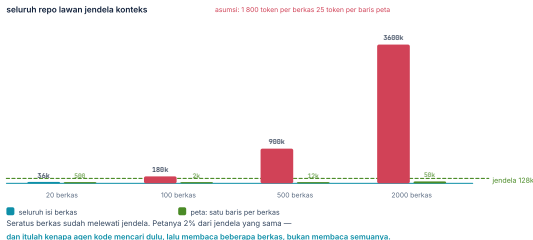
Ini bukti terbaik yang bisa Anda punya untuk angka mana pun yang dilaporkan.

03

Menemukan yang relevan

Sebab seluruh repo tidak akan pernah muat.

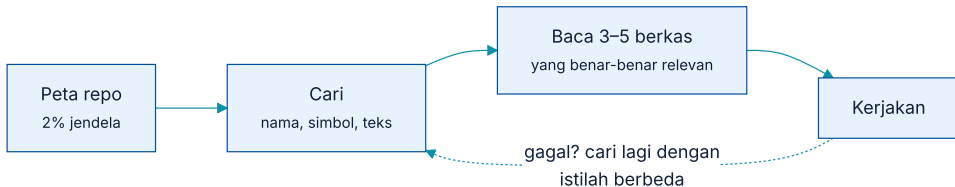
Seratus berkas sudah melewati jendela



Seluruh isi berkas lawan peta satu baris per berkas, dihitung. Langkahi menurut ukuran repo.

\u201cMasukkan saja seluruh kode ke konteks\u201d berhenti bekerja hampir seketika. Petanya dua orde besaran lebih kecil — dan itulah yang membuat pola **peta** → **cari** → **baca beberapa** jadi satu-satunya yang bisa diskalakan.

Peta, cari, baca sedikit



Tiga langkah, dan hanya langkah terakhir yang mahal.

Dua langkah pertama murah dan bisa diulang; hanya langkah ketiga yang memakan konteks. Itu sebabnya urutannya penting.

...dan apa yang dikerjakan tiap langkah

- 1 { 'h': 'Peta memberi tahu apa yang ADA', 'p': 'Nama berkas, kelas, fungsi utama. Cukup untuk memutuskan ke mana harus mencari.' }
- 2 { 'h': 'Pencarian menyempitkan', 'p': 'Berdasarkan nama simbol atau teks — dan di sini pencarian teks biasa **mengalahkan** pencarian kemiripan makna, sebab nama fungsi adalah nama, bukan konsep.' }
- 3 { 'h': 'Baca tiga sampai lima berkas', 'p': 'Itu yang masuk konteks. Sisanya tidak pernah dibaca, dan itu bukan kerugian.' }

Pola yang persis sama dengan Bab 4 dan Bab 9: **jaring lebar dengan yang murah, saringan halus dengan yang mahal**. Ia muncul untuk ketiga kalinya karena alasan yang sama.

Memadatkan konteks tanpa kehilangan yang menentukan

Harus bertahan

Tujuan awal, berkas yang sedang disunting, hasil uji terakhir, dan keputusan yang sudah diambil beserta alasannya.

Boleh hilang

Isi berkas yang sudah dibaca (bisa dibaca ulang), keluaran perintah yang sudah tidak relevan, jalan buntu yang sudah ditinggalkan.

Aturan yang membedakan keduanya sama seperti Bab 4: **apa yang bisa diambil ulang dengan alat boleh dibuang; apa yang hanya ada di percakapan harus dipertahankan.**

Menjaga singgahan tetap hidup di sesi kode

- 1 {'h': 'Taruh yang tetap di depan, dan jangan diutak-atik', 'p': 'Peta repo yang disusun ulang tiap giliran membunuh singgahan tanpa satu pun pesan galat.'}
- 2 {'h': 'Tambahkan di belakang, jangan menyisipkan di tengah', 'p': 'Menyisipkan sesuatu di tengah mengubah awalan semua yang sesudahnya.'}
- 3 {'h': 'Periksa angkanya, jangan percaya', 'p': 'cache_read_input_tokens harus bukan nol. Kalau jadi nol, sesuatu di awalan berubah.'}

Menyunting: tambalan, bukan tulis ulang

CARA	BIAYA TOKEN	RISIKONYA
Kembalikan seluruh berkas Tambalan berupa diff	Besar — dua kali isi berkas Kecil — hanya yang berubah	Bagian yang tidak disentuh bisa ikut berubah tanpa disadari Bisa gagal diterapkan kalau berkasnya bergeser; itu kegagalan yang kelihatan

Baris kedua lebih baik karena alasan yang tidak langsung terlihat: **diff yang gagal diterapkan adalah galat**, sedangkan tulis ulang yang diam-diam mengubah baris lain adalah bug yang lolos ke tinjauan.

Mencari kode: teks mengalahkan makna

Cari teks persis

Nama simbol, pemanggilan, string galat. Cepat, pasti, dan hampir selalu yang benar.

Cari menurut struktur

Definisi, pemanggil, penurunan kelas. Lebih tepat lagi kalau bahasanya mendukung.

Cari kemiripan makna

Berguna untuk "di mana logika kredit?" dan buruk untuk "di mana `hitung_dscr` dipanggil?"

Bentuk yang paling sering benar: **cari teks dulu; kemiripan makna hanya kalau pencarian teks tidak menemukan apa-apa** — dan sebutkan dalam deskripsi alatnya kapan masing-masing dipakai.

Membaca berkas: sebagian, dengan nomor baris

- ① {'h': 'Kembalikan potongan, bukan berkas penuh', 'p': 'Berkas 2 000 baris masuk konteks satu kali dan dibayar berkali kali. Sebagian besar tugas butuh tiga puluh baris.'}
- ② {'h': 'Sertakan nomor barisnya', 'p': 'Supaya tambalan bisa merujuk lokasi, dan supaya jejaknya bisa dibaca ulang.'}
- ③ {'h': 'Sebutkan kalau dipotong', 'p': 'Berkas terpotong yang menyamar sebagai berkas penuh menghasilkan kesimpulan yang salah tentang apa yang ada di dalamnya.'}

Ketiganya pengulangan aturan Bab 5 tentang mengolah keluaran alat, diterapkan pada bentuk data yang paling sering membanjiri konteks agen kode.

04

Dinding

Satu-satunya alat di modul ini yang butuh isolasi sungguhan.

Alat yang bisa menjadi alat apa pun

Karena itu pertanyaan keamanannya berubah bentuk. Untuk alat lain: *apa yang boleh dipanggil?* Untuk alat kode: **di mana ia berjalan, dan apa yang bisa dijangkau dari sana?**

Dan itu pertanyaan infrastruktur, bukan pertanyaan prompt — dijawab dengan proses terpisah, jaringan yang ditutup, dan lingkungan yang tidak memuat kredensial apa pun.

Berlapis, dan jujur soal yang tidak ditutupnya

LAPIS	MENAHAN	TIDAK MENAHAN
Tanpa kredensial di lingkungan Jaringan ditutup	Pemakaian kunci yang bocor Pengiriman data keluar	Kunci yang ada di dalam kode yang dibacanya Apa pun yang sudah ada di dalam sandbox
Batas waktu dan memori	Gelung tak berujung	Kerusakan yang selesai dalam satu detik
Proses / kontainer terpisah	Sebagian besar hal	Celah pada kernel atau salah konfigurasi
Berkas hanya-baca	Perubahan yang tidak disengaja	Pembacaan berkas yang seharusnya tidak terbaca

Kolom ketiga yang membuat tabel ini berguna. **Sandbox yang dijelaskan sebagai \u201ccaman\u201d membuat orang berhenti berpikir**; yang menyebut batasnya membuat orang menaruh data sensitif di tempat lain.

Menjalankan perintah bukan hal yang sama dengan menjalankan kode

Penerjemah kode

Satu bahasa, pustaka yang bisa dibatasi, keluaran yang bentuknya diketahui. Bisa dikurung relatif rapat.

Baris perintah

Seluruh sistem operasi, termasuk hal yang tidak terpikir saat merancang. Daftar perintah yang diizinkan lebih masuk akal daripada daftar yang dilarang.

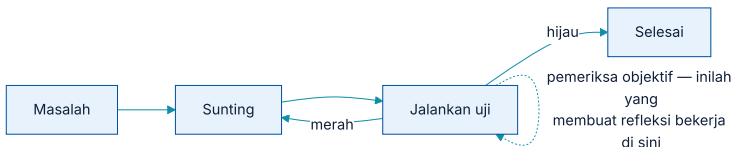
Kalau harus memilih satu untuk dimulai, mulai dari **penerjemah kode tanpa jaringan**. Ia mencakup sebagian besar kegunaan analitis dengan permukaan yang jauh lebih kecil.

05

Untuk rekayasa perangkat lunak

Di sini pemeriksanya sudah ada, dan itu mengubah segalanya.

Gelung yang punya pemeriksa objektif



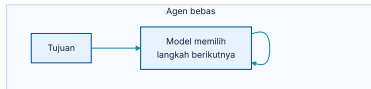
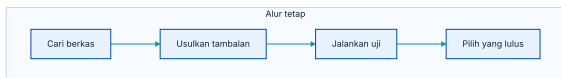
Sunting, jalankan uji, ulangi — dan ujinya yang menentukan berhenti.

Bab 3 dan Bab 6 menyebut syarat refleksi bekerja: harus ada pemeriksa yang lebih mudah daripada menjawab.

Pemrograman adalah domain yang **sudah punya itu sejak awal** — uji yang dijalankan, penyusun yang mengeluh, linter yang menolak.

Itu sebabnya agen kode terasa jauh lebih berhasil daripada agen di domain lain, dan **bukan karena modelnya lebih pandai soal kode**. Ia bekerja di tempat yang jawabannya bisa diperiksa mesin.

Alur tetap sering mengalahkan agen bebas



Empat langkah tetap, lawan gelung yang memilih sendiri.

Untuk perbaikan bug yang bentuknya berulang — temukan berkas, usulkan beberapa tambalan, jalankan uji, ambil yang lulus — **alur tetap sering mengungguli agen bebas**, dengan biaya lebih kecil dan hasil yang bisa diramalkan.

Ini pengulangan pesan Bab 1 pada domain yang paling banyak menghasilkan demo agen: **kalau langkahnya sudah diketahui, yang Anda butuhkan alur tetap**, dan itu tetap berlaku di sini.

Membaca angka tolok ukur kode dengan curiga

Kontaminasi

Repo publik dan isunya kemungkinan besar ada di data latih. Angka tinggi pada isu lama tidak berarti banyak.

Berapa percobaan?

pass@k dengan k besar (Bab 7) membuat angka melonjak tanpa keandalan bertambah.

Ujinya yang menilai

“Berhasil” berarti uji yang ada lulus — bukan bahwa perbaikannya benar. Tambalan yang mematikan ujinya juga lulus.

Bukan repo Anda

Basis kode besar yang lama, tanpa uji, dengan konvensi sendiri adalah masalah yang berbeda.

Basis kode yang sudah ada adalah masalah yang berbeda

Proyek baru

Tidak ada konvensi yang harus dipatuhi, tidak ada kode lama yang bisa rusak, dan uji ditulis bersamaan. Di sinilah demo terlihat mengesankan.

Basis kode yang berumur

Konvensi tak tertulis, kaitan yang tidak terlihat, uji yang menutup sebagian kecil saja. Di sinilah pekerjaan sebenarnya berada.

lebih pandai, melainkan **konteks yang lebih baik**: berkas panduan konvensi, peta modul, dan contoh perubahan serupa yang pernah diterima.

Yang paling menolong pada kolom kanan bukan model yang

Pola yang sama dengan seluruh modul: **perbaiki apa yang masuk ke konteks sebelum mengganti modelnya.**

Perubahan yang menyentuh banyak berkas

- ① {'h': 'Kumpulkan seluruh tambalan dulu, terapkan di akhir', 'p': 'Sama seperti Bab 8: tunda tulisan sampai semuanya siap, supaya kegagalan sebagian tidak meninggalkan keadaan setengah jadi.'}
- ② {'h': 'Jalankan pembangunan dan uji sesudah semuanya diterapkan', 'p': 'Bukan sesudah tiap berkas — itu memberi sinyal yang menyesatkan.'}
- ③ {'h': 'Kalau gagal, kembalikan semuanya', 'p': 'Cabang terpisah membuat ini satu perintah, bukan satu penyelamatan.'}

Meninjau kode yang ditulis agen

Baca diff - nya, bukan ringkasannya

Ringkasan yang ditulis agen menjelaskan apa yang **dimaksudkannya**. Diff menunjukkan apa yang **dilakukannya**.

Periksa apakah ujinya ikut berubah

Uji yang disesuaikan supaya lulus adalah temuan, bukan detail.

Perhatikan yang DIHAPUS

Penanganan galat dan pemeriksaan yang hilang jarang terlihat di ringkasan mana pun.

Tolak perubahan yang terlalu besar

Diff yang tidak bisa dibaca dalam sepuluh menit tidak akan ditinjau dengan sungguh-sungguh — oleh siapa pun.

Kartu terakhir sering menentukan apakah agen kode benar-benar menghemat waktu: **perubahan kecil yang sering mengalahkan perubahan besar yang jarang**, sebab hanya yang pertama benar-benar ditinjau.

Merencanakan sebelum menyunting

- ① {'h': 'Sebutkan berkas yang akan disentuh', 'p': 'Sebelum menyentuhnya. Peninjau bisa menghentikan arah yang salah ketika biayanya masih nol.'}
- ② {'h': 'Sebutkan uji yang harus lulus', 'p': 'Kalau tidak ada ujinya, langkah pertama adalah menulis ujinya.'}
- ③ {'h': 'Sebutkan apa yang TIDAK akan diubah', 'p': 'Ini yang paling menenangkan peninjau, dan paling jarang dituliskan.'}

Bentuk pengawasan yang paling bisa dijalankan tetap sama seperti Bab 6: **satu tinjauan rencana yang benar-benar dibaca, bukan sepuluh persetujuan yang diklik.**

06

Melatih model kode

Satu gelung yang berjalan karena pemeriksanya gratis.

Data buatan yang bisa diperiksa sendiri

- 1 {'h': 'Bangkitkan soal dan calon jawabannya', 'p': 'Dari repo yang ada, atau dari soal yang disusun model sendiri.'}
- 2 {'h': 'Jalankan ujinya', 'p': 'Yang lulus disimpan, yang gagal dibuang. Tidak ada manusia yang perlu menilai.'}
- 3 {'h': 'Latih pada yang lulus, ulangi', 'p': 'Model membaik, soal yang bisa diselesaikannya bertambah, dan putaran berikutnya menghasilkan data yang lebih baik.'}

Bab 3 menyebut batas gelung semacam ini, dan batas itu berlaku di sini juga: **ia hanya sekuat pemeriksanya**, dan soal yang dibangkitkan cenderung mirip yang sudah bisa dijawab.

Kenapa gelung itu berhenti bekerja di luar kode

Punya pemeriksa seperti itu

- ♦ Kode — uji dijalankan
- ♦ Matematika — hasilnya dicocokkan
- ♦ Kueri — dibandingkan dengan jawaban yang diketahui

Tidak punya

- ♦ Menilai kelayakan kredit
- ♦ Meringkas dokumen
- ♦ Menjawab pertanyaan kebijakan

Untuk kolom kanan, yang menggantikan pemeriksa otomatis adalah **kumpulan uji yang ditulis manusia** — Bab 7 — dan tidak ada jalan pintas yang menghindarinya.

Kode yang berjalan bukan kode yang benar

Yang tidak diuji

Perilaku yang tidak punya uji bisa rusak tanpa satu pun lampu merah menyala.

Kasus tepi

Uji yang ada biasanya menutup jalur bahagia. Kasus tepi justru yang paling sering rusak.

Sifat non-fungsional

Kinerja, penggunaan memori, dan keamanan hampir tidak pernah punya uji.

Karena itu tinjauan manusia tetap wajib, dan pertanyaannya bukan "apakah ujinya lulus" melainkan **"apa yang bisa rusak dan tidak akan diketahui"**.

Menyimpan konteks proyek supaya tidak diulang tiap sesi

- 1 {'h': 'Satu berkas panduan di repo', 'p': 'Konvensi, perintah yang dipakai, dan hal yang tidak boleh disentuh. Ditinjau seperti kode, diversikan seperti kode.'}
- 2 {'h': 'Taruh di bagian tetap konteks', 'p': 'Supaya ikut disinggahkan, bukan dikirim ulang sebagai pesan baru tiap kali.'}
- 3 {'h': 'Perbarui ketika agen salah menebak', 'p': 'Tiap kesalahan yang berasal dari konvensi yang tidak tertulis adalah satu baris yang kurang di berkas itu.'}

Langkah ketiga yang membuatnya membaik sendiri: **berkas panduan yang tumbuh dari kesalahan nyata** jauh lebih berguna daripada yang ditulis sekaligus di awal.

07

Risiko dan praktik

Yang khas agen kode, dan tidak muncul pada agen lain.

Empat risiko yang khas agen kode

Rahasia di dalam kode

Agen yang membaca repo membaca juga kunci yang tertinggal di sana — dan bisa memasukkannya ke konteks, lalu ke jejak.

Menyesuaikan uji, bukan kode

Cara tercepat membuat uji hijau adalah mengubah ujinya. Ini terjadi, dan tertangkap hanya kalau diff-nya dibaca.

Perubahan yang merusak diam - diam

Menghapus penanganan galat supaya lulus uji terlihat seperti penyederhanaan.

Ketergantungan yang ditambahkannya

`pip install` sesuatu yang namanya mirip pustaka asli adalah kelas serangan tersendiri.

Keempatnya punya satu penawaran yang sama dan membosankan: **setiap perubahan lewat tinjauan manusia, sebagai diff yang dibaca** — persis seperti kode dari orang lain.

Rahasia: cegah di dua tempat, bukan satu

- 1 {'h': 'Jangan ada rahasia di repo', 'p': 'Ini seharusnya sudah benar sebelum ada agen, dan agen membuat akibatnya lebih cepat terasa.'}
- 2 {'h': 'Saring saat masuk konteks', 'p': 'Pola kunci yang dikenali dibuang sebelum masuk — sebab yang masuk konteks akan masuk jejak.'}
- 3 {'h': 'Saring saat masuk jejak', 'p': 'Jejak disimpan lama dan dibaca banyak orang. Ini tempat kebocoran yang paling sering tidak terpikir.'}

Perhatikan langkah ketiga: **jejak yang dirancang untuk audit bisa berubah jadi tempat penyimpanan rahasia** kalau tidak ada yang menyaringnya.

Urutan memasang agen kode dengan aman

- ① {'h': 'Mulai dari baca saja', 'p': 'Peta, cari, baca. Agen yang menjelaskan basis kode sudah berguna dan tidak bisa merusak apa pun.'}
- ② {'h': 'Tambahkan penerjemah kode tanpa jaringan', 'p': 'Ini membuka sebagian besar kegunaan analitis dengan permukaan terkecil.'}
- ③ {'h': 'Baru sunting berkas, dengan diff yang ditinjau', 'p': 'Dan tidak pernah langsung ke cabang utama.'}
- ④ {'h': 'Baris perintah paling akhir, dengan daftar yang diizinkan', 'p': 'Kalau memang diperlukan. Banyak sistem berhenti sebelum ini dan sudah cukup.'}

Urutan ini sama bentuknya dengan Bab 5: **alat baca dulu, alat tulis satu per satu dengan alasan tertulis**. Yang berubah cuma seberapa tajam alatnya.

Yang ada di demo, dan apa yang jujur disebutkan tentangnya

Itu keputusan penulisan yang disengaja, dan pantas ditiru: pembaca yang diberi tahu batasnya akan menaruh data sensitif di tempat lain. Pembaca yang diberi tahu `\u201ccaman\u201d` tidak akan.

Sama seperti aplikasi selulernya, yang dokumennya menyebut versi Flutter yang dipakai untuk membangunnya dan apa yang belum diuji. **Klaim yang bisa diperiksa mengalahkan klaim yang menenangkan.**

Berapa biayanya, dan di mana biayanya

SUMBER BIAYA	BESAR KALAU	OBATNYA
Berkas yang dibaca	Dibaca penuh, bukan sebagian	Potongan bernomor baris
Keluaran uji	Seluruh keluaran masuk, termasuk yang lulus	Kembalikan yang gagal saja
Peta repo	Repo besar	Peta per direktori, dimuat saat diperlukan
Iterasi	Gelung sunting - uji berputar banyak	Anggaran putaran, dan uji yang lebih cepat

Baris kedua yang paling mudah diperbaiki dan paling sering terlewat: **keluaran uji yang lulus tidak memberi informasi apa pun**, dan bisa ribuan token.

Angka yang memberi tahu agen kode Anda sehat

ANGKA	SEHAT KALAU	GEJALA KALAU TIDAK
Diff yang diterima tanpa perubahan	Naik pelan	Turun → konteksnya memburuk, bukan modelnya
Putaran sunting - uji per tugas	Kecil dan stabil	Naik → ujinya lambat, atau galatnya tidak informatif
Berkas dibaca per tugas	Sedikit	Banyak → pencariannya tidak menemukan yang tepat
Uji yang ikut berubah	No!	Bukan nol → periksa satu per satu, ini bukan detail

Baris terakhir pantas jadi pemeriksaan otomatis: **tandai tiap diff yang menyentuh berkas uji**, dan minta alasan tertulis. Sering sah; kadang tidak.

Kapan agen kode adalah alat yang salah

Keputusan arsitektur

Pilihan yang akibatnya bertahun-tahun dan pertukarannya tidak bisa diuji. Agen bisa menuliskan pilihannya; ia tidak bisa menanggungnya.

Perbaikan darurat di produksi

Tekanan waktu adalah keadaan terburuk untuk meninjau diff dengan sungguh-sungguh.

Kode yang tidak dipahami siapa pun

Kalau tidak ada manusia yang bisa meninjau hasilnya, kecepatannya tidak berarti apa-apa.

Di sinilah ia menang

Perubahan berulang, migrasi mekanis, menulis uji, menjelaskan kode yang sudah ada, dan analisis data.

Sepuluh bab, satu kalimat yang paling sering berlaku

- ① {'h': 'Karena itu bisa diuji', 'p': 'Satu uji yang gagal kalau seseorang menambahkan alat yang melanggar batasnya.'}
- ② {'h': 'Karena itu bisa dijelaskan', 'p': 'Kepada pemeriksa, kepada atasan, kepada orang yang menanggung akibatnya.'}
- ③ {'h': 'Karena itu bertahan', 'p': 'Prompt bisa diubah siapa saja; alat yang tidak ada tetap tidak ada.'}

Sisanya — memori, rencana, refleksi, banyak agen, multi-modal — semuanya menambah kemampuan. **Tidak satu pun mengubah kalimat di atas.**

Yang dibawa pulang dari bab ini

- ❶ {'h': 'Kode mengubah jawaban dari ditebak jadi dihitung', 'p': 'Dan cara menghitungnya tersimpan untuk diperiksa.'}
- ❷ {'h': 'Seluruh repo tidak akan muat; petanya muat', 'p': 'Peta → cari → baca beberapa. Pola yang sama seperti Bab 4 dan 9.'}
- ❸ {'h': 'Ini satu-satunya alat yang butuh dinding sungguhan', 'p': 'Dan dindingnya harus menyebutkan apa yang tidak ditutupnya.'}
- ❹ {'h': 'Ia berhasil karena pemeriksanya sudah ada', 'p': 'Bukan karena modelnya lebih pandai soal kode.'}
- ❺ {'h': 'Alur tetap masih sering menang', 'p': 'Pesan Bab 1, di domain yang paling banyak menghasilkan demo agen.'}